

Day 2's Final Problem: หोजดหมายเหตุแห่งคำร้องที่ถูกกลืน

เรื่องราว

ใต้เมืองดิจิทัลโบราณแห่งนี้ มีสถานที่สำคัญชื่อว่า “หोजดหมายเหตุแห่งคำร้องที่ถูกกลืน” ประชาชนที่ทำข้อมูลสำคัญสูญหายจะต้องมายื่นคำร้องเพื่อขอให้ระบบช่วยกู้คืนข้อมูล โดยคำร้องทุกคำร้องจะต้องถูกจัดการอย่างเป็นระบบและเป็นธรรม

แต่ระบบของหोजดหมายเหตุแห่งนี้ไม่ได้ง่ายเหมือนकिธรรมดาทั่วไป เพราะภายในมีองค์ประกอบสำคัญอยู่ 3 ส่วนที่ทำงานร่วมกัน

ส่วนที่ 1 แฉวรอรับบริการ

คำร้องใหม่ทุกคำร้องจะเข้ามาต่อท้ายแฉวรอ และจะต้องถูกเรียกไปดำเนินการตามลำดับการมาถึงก่อนหลัง

ส่วนที่ 2 ประวัติการย้อนกลับการทำงาน

บางครั้งเจ้าหน้าที่อาจดำเนินการผิดพลาด เช่น รับคำร้องผิด ลบคำร้องผิด หรือย้ายคำร้องผิด ระบบจึงต้องสามารถย้อนกลับ “การกระทำล่าสุด” ได้

ส่วนที่ 3 สายโซ่คำร้องที่กำลังดำเนินการ

เมื่อคำร้องถูกเรียกจากแฉวรอแล้ว คำร้องนั้นจะถูกย้ายเข้าไปเก็บในรายการคำร้องที่กำลังดำเนินการ ซึ่งเก็บในรูปแบบเชื่อมต่อกันเป็นลำดับ

ผู้ออกแบบระบบต้องการโปรแกรมหนึ่งโปรแกรมที่สามารถจำลองทุกส่วนนี้ได้อย่างถูกต้อง

ข้อมูลของคำร้อง

คำร้องแต่ละรายการประกอบด้วยข้อมูลดังนี้

- **id** หมายถึง รหัสคำร้อง เป็นจำนวนเต็ม
- **name** หมายถึง ชื่อผู้ยื่นคำร้อง เป็นสตริงที่ไม่มีช่องว่าง
- **priority** หมายถึง ระดับความสำคัญ เป็นจำนวนเต็ม

คำร้องหนึ่งรายการสามารถอยู่ได้เพียงสถานะใดสถานะหนึ่งต่อไปนี้ในเวลาเดียวกัน

- อยู่ในแถวรอ
- อยู่ในรายการคำร้องที่กำลังดำเนินการ
- ถูกลบออกจากระบบแล้ว

คำสั่งที่ระบบต้องรองรับ

โปรแกรมของนักเรียนต้องรองรับคำสั่งต่อไปนี้

ARRIVE id name priority

มีคำร้องใหม่เข้ามาในระบบ และต้องถูกเพิ่มไว้ท้ายแถวรอ

- id ของคำร้องใหม่จะไม่ซ้ำกับคำร้องที่ยังมีอยู่ในระบบ
- คำสั่งนี้ถือเป็นการกระทำที่ต้องสามารถย้อนกลับได้ และต้องถูกบันทึกไว้ในประวัติการย้อนกลับ

PROCESS

นำคำร้องที่อยู่หน้าสุดของแถวรอออกจากแถวรอ แล้วนำไปต่อท้ายรายการคำร้องที่กำลังดำเนินการ

- ถ้าแถวรอว่าง ให้แสดง NO REQUEST
- คำสั่งนี้ถือเป็นการกระทำที่ต้องบันทึกไว้ในประวัติการย้อนกลับ

CANCEL id

ลบคำร้องที่มีรหัส id ออกจากรายการคำร้องที่กำลังดำเนินการ

- ถ้าไม่พบคำร้องนี้ในรายการคำร้องที่กำลังดำเนินการ ให้แสดง NOT FOUND
- ถ้าพบ ให้ลบคำร้องนั้นออกจากรายการ และแสดง CANCELLED id
- คำสั่งนี้ถือเป็นการกระทำที่ต้องบันทึกไว้ในประวัติการย้อนกลับ

FIND id

ค้นหาคำร้องรหัส id ในรายการคำร้องที่กำลังดำเนินการ

- ถ้าพบ ให้แสดงผลในรูปแบบ
FOUND id name priority position
- โดย position คือ ตำแหน่งของคำร้องในรายการคำร้องที่กำลังดำเนินการ และเริ่มนับจาก 1
- ถ้าไม่พบ ให้แสดง NOT FOUND

NEXT

แสดงข้อมูลของคำร้องที่อยู่หน้าสุดของแถวรอ

- ถ้าแถวรอว่าง ให้แสดง NO REQUEST
- ถ้าไม่ว่าง ให้แสดง
NEXT id name priority

SHOW_ACTIVE

แสดงรายการคำร้องที่กำลังดำเนินการทั้งหมดจากต้นไปท้าย

- ถ้าไม่มีคำร้องที่กำลังดำเนินการ ให้แสดง EMPTY
- ถ้ามี ให้แสดงแต่ละรายการในรูปแบบ
(id,name,priority)
- และคั่นแต่ละรายการด้วย ->

SHOW_WAITING

แสดงคำร้องทั้งหมดที่กำลังรออยู่ในแถวรอจากหน้าไปท้าย

- ถ้าไม่มีคำร้องในแถวรอ ให้แสดง EMPTY
- ถ้ามี ให้แสดงแต่ละรายการในรูปแบบ
(id,name,priority)
- และคั่นแต่ละรายการด้วย |

COUNT_ACTIVE

แสดงจำนวนคำร้องที่กำลังดำเนินการอยู่ในปัจจุบัน

COUNT_WAITING

แสดงจำนวนคำร้องที่กำลังรออยู่ในแถวรอในปัจจุบัน

UNDO

ย้อนกลับ “การกระทำล่าสุด” เฉพาะในกลุ่มคำสั่งต่อไปนี้เท่านั้น

- ARRIVE
- PROCESS
- CANCEL

กติกาของการย้อนกลับมีดังนี้

- ถ้าย้อนกลับคำสั่ง ARRIVE
ให้ลบคำร้องที่เพิ่งถูกเพิ่มล่าสุดนั้นออกจากแถวรอ
- ถ้าย้อนกลับคำสั่ง PROCESS
ให้ลบคำร้องที่เพิ่งถูกย้ายเข้าไปในรายการคำร้องที่กำลังดำเนินการออกมา แล้วนำกลับไปไว้ หน้าที่สุด ของแถวรอ
- ถ้าย้อนกลับคำสั่ง CANCEL
ให้คืนคำร้องนั้นกลับเข้าไปในรายการคำร้องที่กำลังดำเนินการใน ตำแหน่งเดิม ที่มันเคยอยู่ก่อนถูกลบ

ถ้าไม่มีการกระทำใดให้ย้อนกลับ ให้แสดง NO ACTION

ถ้าย้อนกลับสำเร็จ ให้แสดงผลในรูปแบบ

UNDONE action id

โดย action จะเป็นหนึ่งใน

- ARRIVE
- PROCESS
- CANCEL

หมายเหตุสำคัญ

- คำสั่งต่อไปนี้ ไม่ถือเป็นการกระทำที่ต้องบันทึกลงประวัติการย้อนกลับ
 - FIND
 - NEXT
 - SHOW_ACTIVE
 - SHOW_WAITING
 - COUNT_ACTIVE
 - COUNT_WAITING
- ในกรณี UNDO CANCEL คำร้องต้องกลับไปอยู่ในตำแหน่งเดิมในรายการคำร้องที่กำลังดำเนินการ
- ในกรณี UNDO ARRIVE ให้ถือว่าคำร้องนั้นยังไม่ถูกนำไปดำเนินการต่อ และยังคงอยู่ในแถวรอ
- ในกรณี UNDO PROCESS ให้ถือว่าคำร้องที่ถูกย้ายเข้า active ล่าสุดยังคงสามารถย้อนกลับได้อย่างถูกต้อง
- สำหรับตรรกะหลักของโปรแกรม นักเรียน ห้ามใช้ STL container สำเร็จรูป เช่น
 - stack
 - queue
 - list
 - deque
 - vector
- อนุญาตให้ใช้ไลบรารีพื้นฐาน เช่น
 - iostream
 - string

รูปแบบข้อมูลนำเข้า

บรรทัดแรกเป็นจำนวนเต็ม n แทนจำนวนคำสั่งทั้งหมด

จากนั้นอีก n บรรทัด แต่ละบรรทัดเป็นคำสั่งหนึ่งคำสั่งตามรูปแบบที่กำหนด

รูปแบบข้อมูลส่งออก

สำหรับทุกคำสั่งที่ต้องมีการแสดงผล ให้แสดงผลตามที่ระบุไว้ในโจทย์อย่างเคร่งครัด

ข้อจำกัด

- $1 \leq n \leq 2000$
- $1 \leq id \leq 10^9$
- $1 \leq priority \leq 10^6$
- name ไม่มีช่องว่าง
- รหัสคำร้องที่ยังอยู่ในระบบจะไม่ซ้ำกัน

Test case 1	
Input	
16	
ARRIVE 101 Alice 2	
ARRIVE 102 Bob 5	
SHOW_WAITING	
PROCESS	
SHOW_ACTIVE	
NEXT	
PROCESS	
SHOW_ACTIVE	
COUNT_ACTIVE	
CANCEL 101	
SHOW_ACTIVE	
UNDO	
SHOW_ACTIVE	
FIND 102	
COUNT_WAITING	
UNDO	
Output	
(101,Alice,2) (102,Bob,5)	
(101,Alice,2)	
NEXT 102 Bob 5	
(101,Alice,2) -> (102,Bob,5)	
2	
CANCELLED 101	
(102,Bob,5)	
UNDONE CANCEL 101	
(101,Alice,2) -> (102,Bob,5)	
FOUND 102 Bob 5 2	
0	
UNDONE PROCESS 102	

Test case 2	
Input	
10	
PROCESS	
NEXT	
SHOW_ACTIVE	
SHOW_WAITING	
UNDO	
COUNT_ACTIVE	
COUNT_WAITING	
ARRIVE 1 Tom 3	
UNDO	
SHOW_WAITING	
Output	
NO REQUEST	
NO REQUEST	
EMPTY	
EMPTY	
NO ACTION	
0	
0	
UNDONE ARRIVE 1	
EMPTY	

Test case 3	
Input	
14	
ARRIVE 11 Ann 1	
ARRIVE 22 Ben 2	
ARRIVE 33 Cat 3	
PROCESS	
PROCESS	
SHOW_ACTIVE	
SHOW_WAITING	
UNDO	
SHOW_ACTIVE	
SHOW_WAITING	
UNDO	
SHOW_ACTIVE	
SHOW_WAITING	
COUNT_WAITING	
Output	
(11,Ann,1) -> (22,Ben,2)	
(33,Cat,3)	
UNDONE PROCESS 22	
(11,Ann,1)	
(22,Ben,2) (33,Cat,3)	
UNDONE PROCESS 11	
EMPTY	
(11,Ann,1) (22,Ben,2) (33,Cat,3)	
3	

Test case 4	
Input	
15	
ARRIVE 7 A 5	
ARRIVE 8 B 6	
ARRIVE 9 C 7	
PROCESS	
PROCESS	
PROCESS	
SHOW_ACTIVE	
CANCEL 8	
SHOW_ACTIVE	
UNDO	
SHOW_ACTIVE	
FIND 8	
COUNT_ACTIVE	
COUNT_WAITING	
NEXT	
Output	
(7,A,5) -> (8,B,6) -> (9,C,7)	
CANCELLED 8	
(7,A,5) -> (9,C,7)	
UNDONE CANCEL 8	
(7,A,5) -> (8,B,6) -> (9,C,7)	
FOUND 8 B 6 2	
3	
0	
NO REQUEST	

Test case 5	
Input	
13	
ARRIVE 100 X 9	
ARRIVE 200 Y 8	
PROCESS	
CANCEL 100	
UNDO	
UNDO	
SHOW_ACTIVE	
SHOW_WAITING	
COUNT_ACTIVE	
COUNT_WAITING	
NEXT	
FIND 100	
FIND 200	
Output	
CANCELLED 100	
UNDONE CANCEL 100	
UNDONE PROCESS 100	
EMPTY	
(100,X,9) (200,Y,8)	
0	
2	
NEXT 100 X 9	
NOT FOUND	
NOT FOUND	